

第 7 章：查询优化

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

教学内容¹

① 7.1 查询优化的基本概念

② 7.2 逻辑查询优化

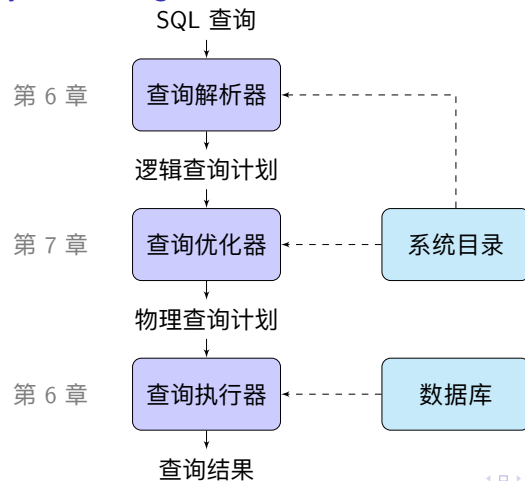
- 7.2.1 逻辑查询计划的代价估计
- 7.2.2 逻辑查询计划枚举
- 7.2.3 连接顺序优化

③ 7.3 物理查询优化

- 7.3.1 物理算子的确定
- 7.3.2 数据传递方式的确定

¹课件更新于 2026 年 5 月 25 日

查询处理 (Query Processing) 的基本过程



查询优化技术的诞生

- 20 世纪 70 年代, IBM System R 最早实现了 RDBMS 的查询优化器
- System R 的查询优化器的很多概念和技术仍被现在的 DBMS 使用



Patricia Selinger

System R 的主要团队成员

基于代价的查询优化 (Cost-based Query Optimization) 的创造者

7.1 查询优化的基本概念

查询计划的代价 (Query Plan Cost)

查询计划的代价是执行该计划中所有算子的执行代价之和

- 代价是反映查询计划执行时间长短的数
- 查询计划的代价越低，其执行时间越短
- 查询计划的代价与实际执行时间没有任何直接的函数关系
- 比较不同查询的查询计划代价没有意义

例 (PostgreSQL 查询优化器的代价常量)

- 顺序读一个磁盘页的代价: $\text{seq_page_cost} = 1.0$
- 随机读一个磁盘页的代价: $\text{random_page_cost} = 4.0$
- CPU 处理一条元组的代价: $\text{cpu_tuple_cost} = 0.01$
- CPU 处理一个索引项的代价: $\text{cpu_index_tuple_cost} = 0.005$
- CPU 处理一个操作或函数的代价: $\text{cpu_operator_cost} = 0.0025$
- ...

基于代价的查询优化 (Cost-based Query Optimization)

求与初始查询计划 P_0 等价且代价最低的查询计划

1. 查询计划枚举 (Query Plan Enumeration)

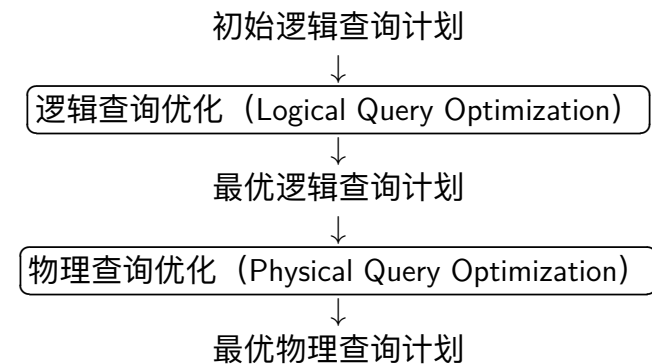
不断生成与 P_0 等价的查询计划 P



2. 查询代价估计 (Query Cost Estimation)

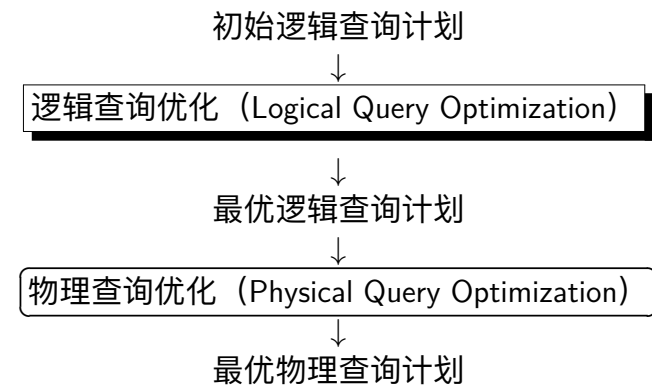
对每个生成的查询计划 P , 计算 P 的代价

查询优化的两个阶段



7.2 逻辑查询优化

查询优化的两个阶段



基于代价的逻辑查询优化

1. 逻辑查询计划枚举 (Plan Enumeration)

不断生成与初始逻辑查询计划 P_0 等价的逻辑查询计划 P



2. 逻辑查询代价估计 (Cost Estimation)

对每个生成的逻辑查询计划 P , 计算 P 的代价

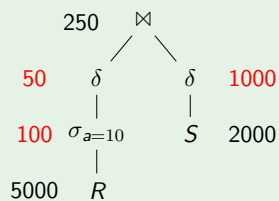
7.2.1 逻辑查询计划的代价估计

逻辑查询计划的简易代价模型

逻辑查询计划的代价：查询计划执行过程中产生的**中间结果的元组总数**

- ① 查询计划的执行时间 $\leftarrow \text{estimate}$ 查询计划执行过程中产生的 I/O 次数
- ② I/O 次数 $\leftarrow \text{estimate}$ 查询计划执行过程中产生的中间结果大小
- ③ 中间结果大小 $\leftarrow \text{estimate}$ 中间结果的基数 (Cardinality)，即元组数

例 (逻辑查询计划的代价)



$$\text{代价} = 50 + 100 + 1000 = 1150$$

RDBMS 查看估计基数

EXPLAIN SELECT 语句;

```
D EXPLAIN SELECT 学号, 姓名 FROM students WHERE 专业 = '人工智能';
```

Physical Plan
SEQ_SCAN
Table: students
Type: Sequential Scan
Projections:
学号
姓名
Filters:
专业='人工智能'
~105 rows

RDBMS 查看实际基数

EXPLAIN ANALYZE SELECT 语句;

```
D EXPLAIN ANALYZE SELECT 学号, 姓名 FROM students WHERE 专业 = '人工智能';
```

Query Profiling Information
EXPLAIN ANALYZE SELECT 学号, 姓名 FROM students WHERE 专业 = '人工智能';
Total Time: 0.0013s
QUERY
EXPLAIN_ANALYZE
0 rows (0.00s)
TABLE_SCAN
Table: students
Type: Sequential Scan
Projections:
学号
姓名
Filters:
专业='人工智能'
28 rows (0.00s)

逻辑查询计划的代价模型 vs 物理查询计划的代价模型

物理查询计划中包含一些注释

- 算子的实现算法
- 中间结果的传递方式 (物化或流水线)
- 中间结果是否排序

可以根据注释分析执行每个算子的**I/O 代价**和**CPU 计算代价**

- 物理查询计划的代价比逻辑查询计划的代价更可靠
- DBMS 实际上直接枚举物理查询计划，而不区分逻辑查询优化和物理查询优化

无论如何，代价模型一定需要估计每个操作的结果元组数，即**基数估计 (cardinality estimation)**

基数估计 (Cardinality Estimation)

基数估计: 估计查询结果的元组数

查询优化器对基数估计方法的要求

- 准确
- 易计算
- 逻辑一致 (Logically Consistent)

逻辑一致性 (Logically Consistency)

- 单调性: 一个操作的输入越大, 操作结果的基数估计值越大
- 顺序无关性: 在多个关系上执行同一种满足交换律和结合律的操作时 (如 $\bowtie$, $\times$, $\cup$, $\cap$), 最终结果基数的估计值与操作执行顺序无关

基数估计所需的数据库统计信息

系统目录 (System Catalog) 中记录着基数估计所需的数据库统计信息

- $T(R)$: 关系 R 的元组数
- $V(R, A)$: 关系 R 的属性集 A 的不同值的个数

基本假设 (过于理想)

- ① 关系中每个属性的取值均服从均匀分布
- ② 关系的所有属性相互独立

例 (数据库统计信息)

X	Y
a	1
b	2
a	2
b	1
c	1
c	2

- $T(R) = 6$, $V(R, X) = 3$, $V(R, Y) = 2$
- X 均匀分布, Y 均匀分布
- X 和 Y 独立

收集数据库统计信息

DBMS 自动收集

用户手动收集

- Postgres/SQLite: **ANALYZE**
- Oracle/MySQL: **ANALYZE TABLE**
- SQL Server: **UPDATE STATISTICS**
- DB2: **RUNSTATS**

笛卡尔积操作的基数估计

笛卡尔积操作的基数估计

$$T(R \times S) = T(R) T(S)$$

例 (笛卡尔积操作的基数估计)

X
a
b
c

$\times$

Y
1
2

$=$

X	Y
a	1
b	2
a	2
b	1
c	1
c	2

投影操作的基数估计

不含去重的投影操作的基数估计

$$T(\Pi_L(R)) = T(R)$$

含去重的投影操作的基数估计

$$T(\Pi_L(R)) = V(R, L)$$

例 (投影操作的基数估计)

R	
X	Y
a	1
b	2
a	2
b	1
c	1
c	2

$\Pi_X(R)$ (不含去重)

X
a
b
a
b
c
c

$\Pi_X(R)$ (含去重)

X
a
b
c

选择操作的基数估计

$$S = \sigma_{A=c}(R)$$

- $T(S) = T(R)/V(R, A)$ (基于均匀分布假设)

$$S = \sigma_{A \neq c}(R)$$

- $T(S) = T(R)$ 或
- $T(S) = T(R) - T(R)/V(R, A)$

例 (选择操作的基数估计)

R	
X	Y
a	1
b	2
a	2
b	1
c	1
c	2

$\sigma_{X=a}(R)$	
X	Y
a	1
a	2

$\sigma_{X \neq a}(R)$	
X	Y
b	2
b	1
c	1
c	2

选择操作的基数估计

$$S = \sigma_{A>c}(R) \text{ or } S = \sigma_{A<c}(R)$$

- $T(S) = T(R)/3$ (《数据库系统实现 (第 2 版)》) 或
- $T(S) = T(R)/2$ (《数据库系统概念 (第 6 版)》)
- 如果系统目录中记录了 A 属性的最大值 h_A 和最小值 l_A , 则 $T(\sigma_{A>c}(R)) = T(R) \frac{h_A - c}{h_A - l_A}$ (基于均匀分布假设)

例 (选择操作的基数估计)

R	
X	Y
a	1
b	2
a	2
b	1
c	1
c	2

$\sigma_{Y>1.5}(R)$

X	Y
b	2
a	2
c	2

选择操作的基数估计

$$S = \sigma_{\theta_1 \wedge \theta_2}(R)$$

- $S = \sigma_{\theta_1 \wedge \theta_2}(R) = \sigma_{\theta_1}(\sigma_{\theta_2}(R)) = \sigma_{\theta_2}(\sigma_{\theta_1}(R))$
- $T(S) = T(R)f_1f_2$ (假设 θ_1 和 θ_2 独立)

$$f_i = \begin{cases} 1/V(R, A) & \text{if } \theta_i \text{ is of the form } A = c, \\ 1 - 1/V(R, A) & \text{if } \theta_i \text{ is of the form } A \neq c, \\ 1/3 & \text{if } \theta_i \text{ is of the form } A < c \text{ or } A > c, \end{cases}$$

例 (选择操作的基数估计)

R	
X	Y
a	1
b	2
a	2
b	1
c	1
c	2

$\sigma_{X=a}(R)$	
X	Y
a	1
a	2

$\sigma_{Y=1}(\sigma_{X=a}(R))$	
X	Y
a	1

选择操作的基数估计

$$S = \sigma_{\theta_1 \vee \theta_2}(R)$$

- $S = \sigma_{\theta_1}(R) \cup \sigma_{\theta_2}(R) = R - \sigma_{\neg\theta_1 \wedge \neg\theta_2}(R)$
- $T(S) = T(R)(1 - (1 - f_1)(1 - f_2))$

$$S = \sigma_{\neg\theta}(R)$$

- $S = R - \sigma_{\theta}(R)$
- $T(S) = T(R) - T(\sigma_{\theta}(R))$

例 (选择操作的基数估计)

R	
X	Y
a	1
b	2
a	2
b	1
c	1
c	2

$\sigma_{X \neq a \wedge Y \neq 1}(R)$	
X	Y
b	2
c	2

$\sigma_{X=a \vee Y=1}(R)$	
X	Y
a	1
b	1
a	2
c	1

邹兆年 (CS@HIT)

第 7 章: 查询优化

2026 春

25 / 115

二路自然连接 (2-Way Natural Join) 的基数估计

考虑两个关系 R 和 S 的自然连接 $R \bowtie S$

基本假设

- 连接属性值集合包含假设 (Containment of Value Sets)**
对于连接属性 K , 如果 $V(R, K) \leq V(S, K)$, 则 $R.K$ 的属性值集合是 $S.K$ 的属性值集合的子集
- 非连接属性值集合保留假设 (Preservation of Value Sets)**
对于 R 中任意非连接属性 A , 有 $V(R \bowtie S, A) = V(R, A)$

例 (连接基数估计的基本假设)

R	
X	Y
1	a
2	b
2	a
1	b
1	c
2	c

$\bowtie$

S	
Y	Z
a	α
b	β

=

X	Y	Z
1	a	α
2	b	β
2	a	α
1	b	β

邹兆年 (CS@HIT)

第 7 章: 查询优化

2026 春

26 / 115

二路自然连接的基数估计

情况 1: R 和 S 只有一个连接属性 Y

$$T(R \bowtie S) = \frac{T(R) T(S)}{\max(V(R, Y), V(S, Y))}$$

例 (二路连接的基数估计)

$R(a, b)$	$S(b, c)$	$U(c, d)$
$T(R) = 1000$	$T(S) = 2000$	$T(U) = 5000$
$V(R, b) = 20$	$V(S, b) = 50$	
	$V(S, c) = 100$	$V(U, c) = 500$

- $T((R \bowtie S) \bowtie U) = \frac{1000 \times 2000 \times 5000}{50} = 400000$
- $T(R \bowtie (S \bowtie U)) = \frac{1000 \times 2000 \times 5000}{50} = 400000$

邹兆年 (CS@HIT)

第 7 章: 查询优化

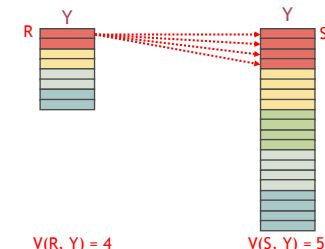
2026 春

27 / 115

证明

不失一般性, 假设 $V(R, Y) \leq V(S, Y)$

- 根据属性值包含假设, $R.Y$ 的属性值集合是 $S.Y$ 的属性值集合的子集, 因此 R 的每个元组都可以和 S 的一些元组进行连接
- 根据属性值均匀分布假设, R 的每个元组都可以和 S 中 $\frac{T(S)}{V(S, Y)}$ 个元组进行连接
- 因此, $R \bowtie S$ 的结果包含 $\frac{T(R) T(S)}{V(S, Y)}$ 个元组



$V(R, Y) = 4$

$V(S, Y) = 5$

邹兆年 (CS@HIT)

第 7 章: 查询优化

2026 春

28 / 115

二路自然连接的基数估计

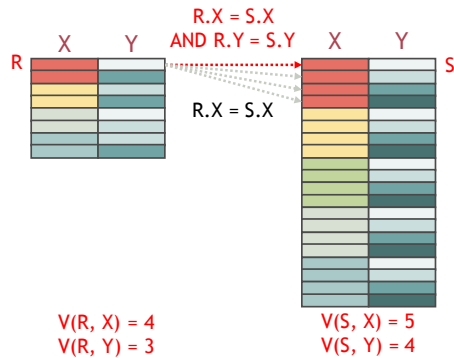
情况 2: R 和 S 有 2 个连接属性 X 和 Y

$$T(R \bowtie S) = \frac{T(R)T(S)}{\max(V(R, X), V(S, X)) \max(V(R, Y), V(S, Y))}$$

例 (二路自然连接的基数估计 (2 个连接属性))

R					$R \bowtie_{R.X=S.X, S} S$				
X	Y				R.X	S.X	R.Y	S.Y	Z
1	a				1	1	a	a	α
2	b				1	1	a	b	β
2	a				1	1	b	a	α
1	b				1	1	b	b	β
1	c				1	1	c	a	α
2	c				1	1	c	b	β

证明



证明

设 $U = R \bowtie_{R.X=S.X} S$, 则 $T(R \bowtie S) = T(\sigma_{R.Y=S.Y}(U))$

- $T(U) = \frac{T(R)T(S)}{\max(V(R, X), V(S, X))}$
- 根据属性值保留假设, 有 $V(U, R.Y) = V(R, Y)$, $V(U, S.Y) = V(S, Y)$
- 不失一般性, 假设 $V(R, Y) \leq V(S, Y)$, 则 $V(U, R.Y) \leq V(U, S.Y)$
- 根据属性值包含假设, U 的 $R.Y$ 属性值集合是 U 的 $S.Y$ 属性值集合的子集
- 根据属性值均匀分布假设, U 中满足 $R.Y = S.Y$ 的元组占 $1/V(U, S.Y) = 1/V(S, Y)$
- 因此, $T(\sigma_{R.Y=S.Y}(U)) = \frac{T(R)T(S)}{\max(V(R, X), V(S, X)) \cdot V(S, Y)}$

二路自然连接的基数估计

例 (二路连接的基数估计)

$R(a, b)$	$S(b, c)$	$U(c, d)$
$T(R) = 1000$	$T(S) = 2000$	$T(U) = 5000$
$V(R, b) = 20$	$V(S, b) = 50$	
	$V(S, c) = 100$	$V(U, c) = 500$

- $T((R \bowtie S) \bowtie U) = \frac{1000 \times 2000 \times 5000}{50 \times 500} = 400000$
- $T(R \bowtie (S \bowtie U)) = \frac{1000 \times \frac{2000 \times 5000}{500}}{50} = 400000$
- $T((R \bowtie U) \bowtie S) = \frac{(1000 \times 5000) \times 2000}{50 \times 500} = 400000$
- 基数估计结果与连接顺序无关

二路自然连接的基数估计

情况 3: R 和 S 有 2 个以上连接属性

证明留作课后习题

多路自然连接 (Multi-Way Natural Join) 的基数估计

设 $S = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$

$$T(S) = \frac{T(R_1) T(R_2) \dots T(R_n)}{\prod_{A \in \{\text{attributes appearing in more than two relations}\}} V(A)}$$

设 $R_{i_1}, R_{i_2}, \dots, R_{i_n}$ 是所有包含属性 A 的关系, 且 $V(R_{i_1}, A) \leq V(R_{i_2}, A) \leq \dots \leq V(R_{i_n}, A)$

$$V(A) = V(R_{i_2}, A) V(R_{i_3}, A) \dots V(R_{i_n}, A)$$

自然连接操作基数估计方法的逻辑一致性

无论是按不同的顺序进行多次二路连接, 还是进行一次多路连接, 上述基数估计方法得到的结果均相同

例 (连接操作的基数估计)

$R(a, b)$	$S(b, c)$	$U(c, d)$
$T(R) = 1000$	$T(S) = 2000$	$T(U) = 5000$
$V(R, b) = 20$	$V(S, b) = 50$	
	$V(S, c) = 100$	$V(U, c) = 500$

- $T((R \bowtie S) \bowtie U) = \frac{1000 \times 2000 \times 5000}{50} = 400000$
- $T(R \bowtie (S \bowtie U)) = \frac{1000 \times \frac{2000 \times 5000}{50}}{50} = 400000$
- $T((R \bowtie U) \bowtie S) = \frac{(1000 \times 5000) \times 2000}{50 \times 500} = 400000$
- $T(R \bowtie S \bowtie U) = \frac{1000 \times 5000 \times 2000}{50 \times 500} = 400000$

集合并操作的基数估计

集合并的基数估计

$$T(R \cup S) = \frac{1}{2}(\max(T(R), T(S)) + T(R) + T(S))$$

- 理由: $\max(T(R), T(S)) \leq T(R \cup S) \leq T(R) + T(S)$

集合差操作的基数估计

集合差的基数估计

$$T(R - S) = T(R) - T(S)/2$$

- 理由: $T(R) - T(S) \leq T(R - S) \leq T(R)$

集合交操作的基数估计

集合交的基数估计 1

$$T(R \cap S) = \min(T(R), T(S))/2$$

- 理由: $0 \leq T(R \cap S) \leq \min(T(R), T(S))$

集合交的基数估计 2

$$T(R \cap S) = T(R \bowtie S)$$

- 理由: $R \cap S = R \bowtie S$

去重操作的基数估计

去重的基数估计 1

$$T(\delta(R)) = (T(R) + 1)/2 \approx T(R)/2$$

- 理由: $1 \leq T(\delta(R)) \leq T(R)$

去重的基数估计 2

$$T(\delta(R)) = \min((T(R) + 1)/2, V(R, A_1) V(R, A_2) \cdots V(R, A_n))$$

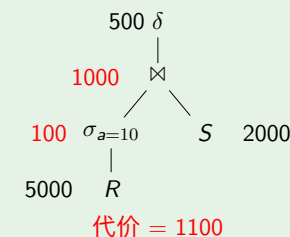
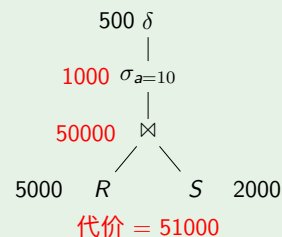
- $A_1, A_2, \dots, A_n$ 是 R 的属性
- 理由: $1 \leq T(\delta(R)) \leq V(R, A_1) V(R, A_2) \cdots V(R, A_n)$

逻辑查询计划的代价估计

例 (逻辑查询计划的代价估计)

- $R(a, b) : T(R) = 5000, V(R, a) = 50, V(R, b) = 100$

- $S(b, c) : T(S) = 2000, V(S, b) = 200, V(S, c) = 100$



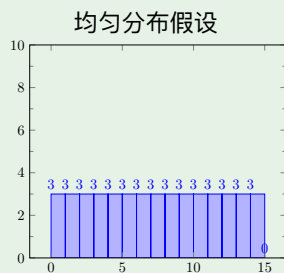
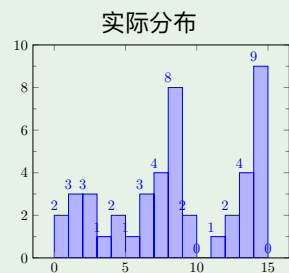
属性值分布的精确近似

真实数据经常不满足均匀分布假设，导致基数估计误差较大

例 (均匀分布假设对基数估计的影响)

属性值 ≥ 13 的元组个数

- 实际情况：13 个
- 均匀分布假设：6 个



直方图 (Histogram)

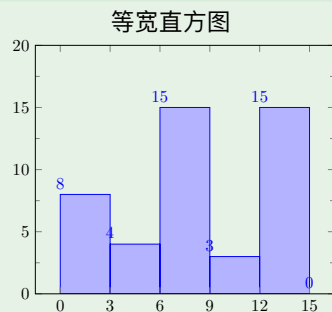
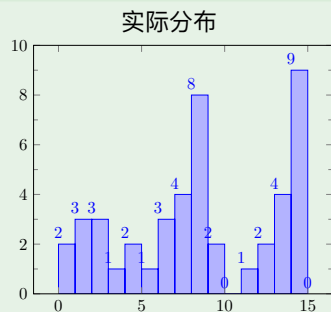
DBMS 使用直方图来记录属性值在不同区间内的出现频率，用于近似数据分布

- 等宽直方图 (Equal-Width Histogram)
- 等高直方图 (Equal-Height Histogram)
- 压缩直方图 (Compressed Histogram)

等宽直方图 (Equal-Width Histogram)

- 属性值各区间的宽度相同
- 各区间内属性值的出现次数不同

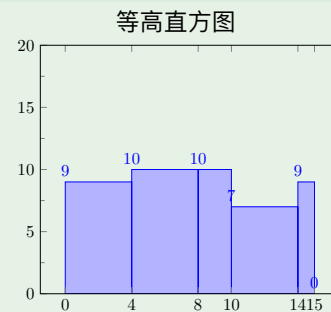
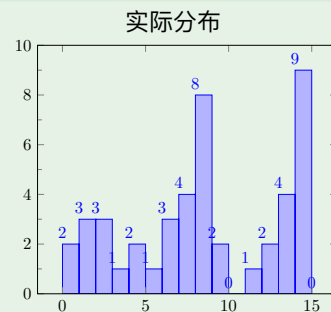
例 (等宽直方图)



等高直方图 (Equal-Height Histogram)

- 属性值各区间的宽度不同
- 各区间内属性值的出现次数基本相同

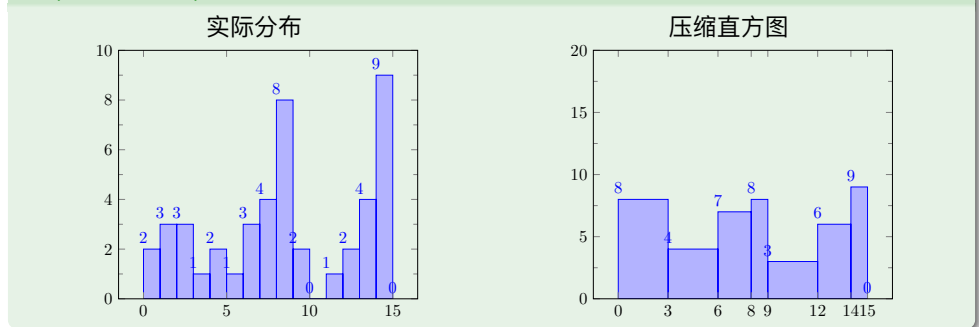
例 (等高直方图)



压缩直方图 (Compressed Histogram)

- 单独记录少量频繁出现的属性值
- 用等高或等宽直方图近似其他属性值的分布

例 (压缩直方图)

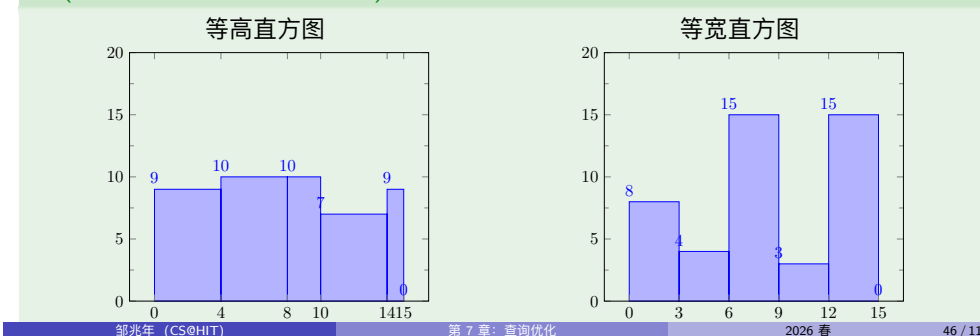


等高直方图 vs 等宽直方图

大多数 DBMS 使用等高直方图, 而非等宽直方图

- 包含高频属性值的区间窄, 对高频属性值的频率估计更准确 (高频属性值对基数估计的误差影响更大)
- 区间边界由数据确定, 无需用户确定

例 (等高直方图 vs 等宽直方图)



PostgreSQL 显示直方图区间的边界

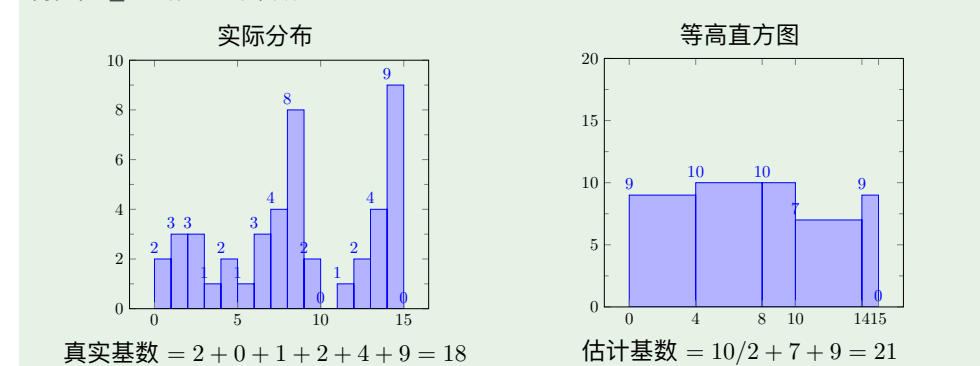
```
SELECT attname, histogram_bounds
FROM pg_stats
WHERE tablename = 'your_table_name' AND attname = 'your_column_name';
```

基于直方图的基数估计

假设: 直方图中每个区间内的属性值服从均匀分布

例 (基于直方图的基数估计)

属性值 ≥ 9 的元组的个数



基于直方图的连接结果基数估计

例 (基于直方图的连接结果基数估计)

已知关系 $R(a, b)$ 和 $S(b, c)$ 的属性 $R.b$ 和 $S.b$ 的直方图如下:

Range	$R.b$	$S.b$
0-9	40	0
10-19	60	0
20-29	80	0
30-39	50	0
40-49	10	5
50-59	5	20
60-69	0	50
70-79	0	100
80-89	0	60
90-99	0	10
Total	245	245

- 使用直方图: $T(R \bowtie S) = 10 \times 5/10 + 5 \times 20/10 = 15$
- 不使用直方图: $T(R \bowtie S) = 245 \times 245/100 = 600$

属性相关性 (Attribute Correlation)

当数据不满足属性独立性假设时, 上述基数估计方法的误差较大

例 (属性独立性假设存在的问题)

设关系 $R(A, B)$ 中属性 A 和 B 满足约束条件 $B = A + 1$, 已知

$T(R) = 1000, V(R, A) = 10, V(R, B) = 10$

- 属性独立: $T(\sigma_{A=1 \wedge B=2}(R)) = T(\sigma_{A=1}(\sigma_{B=2}(R))) = 1000 \times \frac{1}{10} \times \frac{1}{10} = 10$
- 属性相关: $T(\sigma_{A=1 \wedge B=2}(R)) = T(\sigma_{A=1}(\sigma_{B=2}(R))) = 1000 \times \frac{1}{10} \times 1 = 100$

基数估计的本质

- 将属性视作随机变量
- 估计随机变量的联合概率密度函数或概率质量函数

探究式学习: 刻画属性相关性的方法

- 多维直方图 (Multi-Dimensional Histogram)
- 概率图模型 (Probabilistic Graphical Model)
- 神经网络 (Neural Network)
- 核密度估计 (Kernel Density Estimation, KDE)
- 和积网络 (Sum-Product Network, SPN)
- 自回归模型 (Autoregressive Model)



H. Lan, Z. Bao, Y. Peng. **A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration.** *Data Science and Engineering*, 6:86–101, 2021.

Section 3 “Cardinality Estimation”

7.2.2 逻辑查询计划枚举

逻辑查询计划枚举

逻辑查询计划的等价变换

- 将一个逻辑查询计划转换为等价的逻辑查询计划

连接顺序优化

- 确定连接操作的最优执行顺序

等价逻辑查询计划

定义 (等价逻辑查询计划)

如果两个逻辑查询计划在数据库的任意实例上都有相同的查询结果，则这两个逻辑查询计划等价

例 (等价逻辑查询计划)

初始逻辑查询计划

$\sigma_{Sage>18}$
|
 $\Pi_{Sno,Sage}$
|
Student

$\equiv$

等价逻辑查询计划

$\Pi_{Sno,Sage}$
|
 $\sigma_{Sage>18}$
|
Student

逻辑查询计划的等价变换规则

既满足交换律又满足结合律的关系代数操作

- 1 $R \times S = S \times R$
 $(R \times S) \times T = R \times (S \times T)$
- 2 $R \bowtie S = S \bowtie R$
 $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$
- 3 $R \cap S = S \cap R$
 $(R \cap S) \cap T = R \cap (S \cap T)$
- 4 $R \cup S = S \cup R$
 $(R \cup S) \cup T = R \cup (S \cup T)$

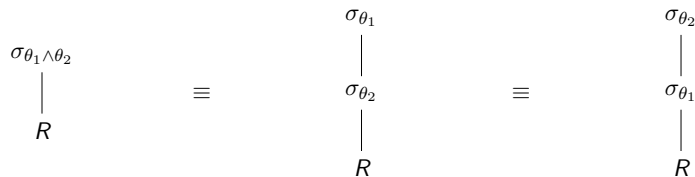
θ 连接满足交换律，但不满足结合律

- $(R(a, b) \bowtie_{R.b>S.b} S(b, c)) \bowtie_{a<d} T(c, d) \neq R(a, b) \bowtie_{R.b>S.b} (S(b, c) \bowtie_{a<d} T(c, d))$

有关选择的等价变换规则 1

选择操作的分裂法则 1

$$\sigma_{\theta_1 \wedge \theta_2}(R) = \sigma_{\theta_1}(\sigma_{\theta_2}(R)) = \sigma_{\theta_2}(\sigma_{\theta_1}(R))$$



有关选择的等价变换规则 2

选择操作的分裂法则 2

$$\sigma_{\theta_1 \vee \theta_2}(R) = \sigma_{\theta_1}(R) \cup \sigma_{\theta_2}(R)$$



有关选择的等价变换规则 3

将选择操作推至集合并操作之下

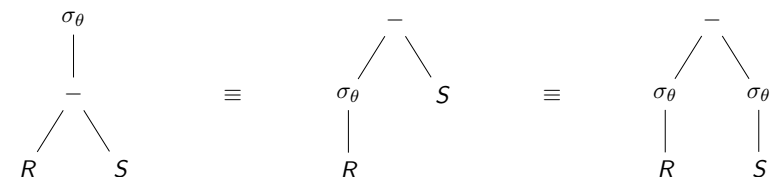
$$\sigma_{\theta}(R \cup S) = \sigma_{\theta}(R) \cup \sigma_{\theta}(S)$$



有关选择的等价变换规则 4

将选择操作推至集合差操作之下

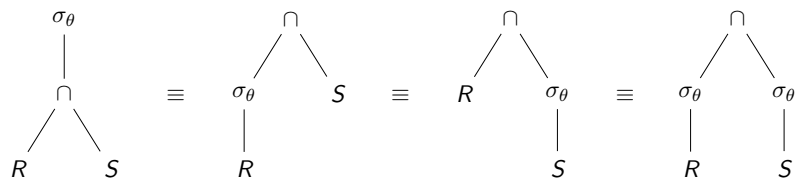
$$\sigma_{\theta}(R - S) = \sigma_{\theta}(R) - S = \sigma_{\theta}(R) - \sigma_{\theta}(S)$$



有关选择的等价变换规则 5

将选择操作推至集合交操作之下

$$\sigma_{\theta}(R \cap S) = \sigma_{\theta}(R) \cap S = R \cap \sigma_{\theta}(S) = \sigma_{\theta}(R) \cap \sigma_{\theta}(S)$$



有关选择的等价变换规则 6

将选择操作推至笛卡尔积操作之下 (一般情况)

$$\sigma_{\theta}(R \times S) = R \bowtie_{\theta} S$$



有关选择的等价变换规则 7

将选择操作推至笛卡尔积操作之下 (特殊情况)

如果 R 中包含 θ 中全部属性, 则 $\sigma_{\theta}(R \times S) = \sigma_{\theta}(R) \times S$



有关选择的等价变换规则 8

将选择操作推至 θ 连接操作之下 (一般情况)

$$\sigma_{\theta_1}(R \bowtie_{\theta_2} S) = R \bowtie_{\theta_1 \wedge \theta_2} S$$



有关选择的等价变换规则 9

将选择操作推至 θ 连接操作之下 (特殊情况)

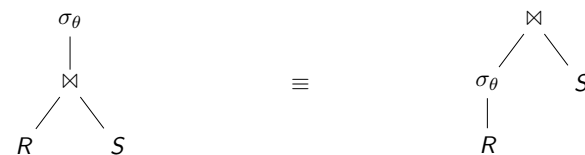
如果 R 中包含 θ_1 中全部属性, 则 $\sigma_{\theta_1}(R \bowtie_{\theta_2} S) = \sigma_{\theta_1}(R) \bowtie_{\theta_2} S$



有关选择的等价变换规则 10

将选择操作推至自然连接操作之下

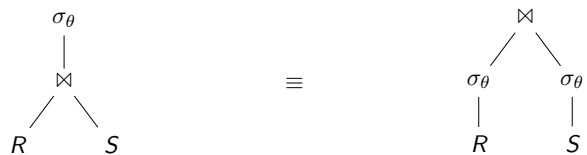
如果 R 中包含 θ 中全部属性, 则 $\sigma_{\theta}(R \bowtie S) = \sigma_{\theta}(R) \bowtie S$



有关选择的等价变换规则 11

将选择操作推至自然连接操作之下 (情况 2)

如果 R 和 S 中均包含 θ 中全部属性, 则 $\sigma_{\theta}(R \bowtie S) = \sigma_{\theta}(R) \bowtie \sigma_{\theta}(S)$

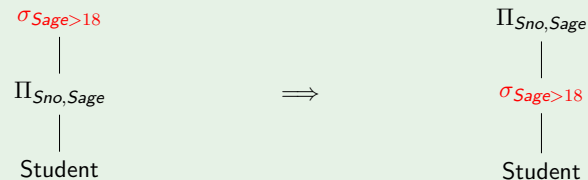


选择下推 (Selection Pushdown) 1

将关系代数表达式树中的选择操作下推 (Push Down), 通常可以提高查询执行效率

- 选择下推可以尽早过滤掉与结果无关的元组

例 (选择下推 1)

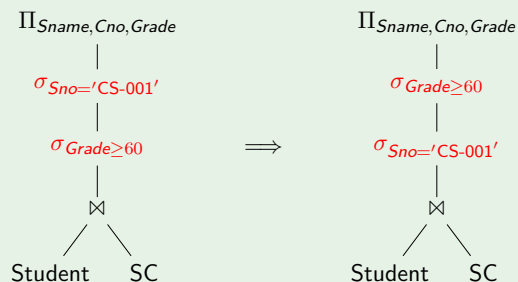


选择下推 2

选择度高的选择操作优先做

- 选择度 (Selectivity): 满足选择条件的元组所占的比例 (比例越低, 选择度越高)
- 尽早过滤掉更多与结果无关的元组

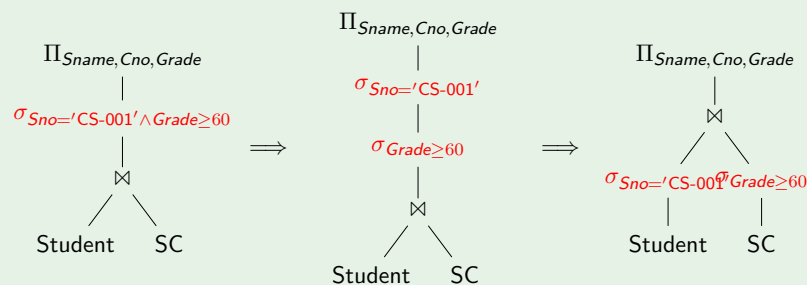
例 (选择下推 2)



选择下推 3

先分解合取选择条件, 再进行选择下推

例 (选择下推 3)

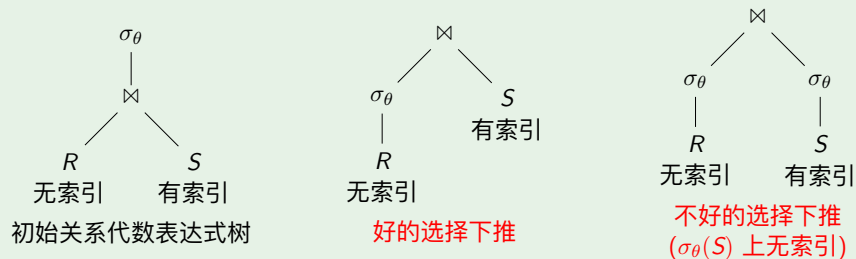


选择下推 4

当选择操作可以向关系代数表达式树的多个分支下推时, 需要考虑向哪个分支下推更合适

- 索引会影响选择下推的方案
- 选择操作的结果上没有索引

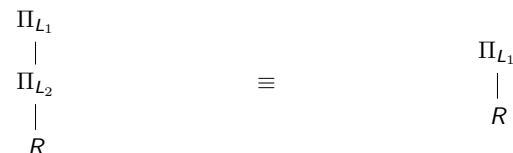
例 (索引对选择下推的影响)



有关投影的等价变换规则 1

将投影操作推至投影操作之下 (吸收律)

$$\Pi_{L_1}(\Pi_{L_2}(R)) = \Pi_{L_1}(R) \quad (L_1 \subseteq L_2)$$



有关投影的等价变换规则 2

将投影操作推至选择操作之下

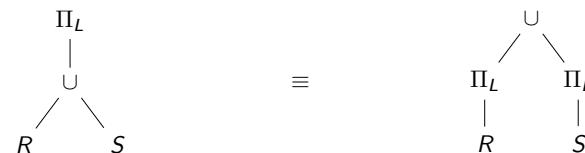
$\Pi_L(\sigma_\theta(R)) = \Pi_L(\sigma_\theta(\Pi_M(R)))$, 其中 M 既包含 L 中全部属性, 又包含 θ 中全部属性



有关投影的等价变换规则 3

将投影操作推至集合并操作之下

$$\Pi_L(R \cup S) = \Pi_L(R) \cup \Pi_L(S)$$



将集合并换作集合交或集合差, 等价变换规则还成立吗?

有关投影的等价变换规则 4

将投影操作推至笛卡尔积操作之下

$$\Pi_L(R \times S) = \Pi_L(\Pi_M(R) \times \Pi_N(S))$$

- M 中包含既在 R 中, 又在 L 中的属性
- N 中包含既在 S 中, 又在 L 中的属性



有关投影的等价变换规则 5

将投影操作推至自然连接操作之下

$$\Pi_L(R \bowtie S) = \Pi_L(\Pi_M(R) \bowtie \Pi_N(S))$$

- M 中包含既在 R 中, 又在 L 中的属性, 以及 R 中的连接属性
- N 中包含既在 S 中, 又在 L 中的属性, 以及 S 中的连接属性



有关投影的等价变换规则 6

将投影操作推至 θ 连接操作之下

$$\Pi_L(R \bowtie_{\theta} S) = \Pi_L(\Pi_M(R) \bowtie_{\theta} \Pi_N(S))$$

- M 中包含既在 R 中, 又在 L 中的属性, 以及 R 中的连接属性
- N 中包含既在 S 中, 又在 L 中的属性, 以及 S 中的连接属性

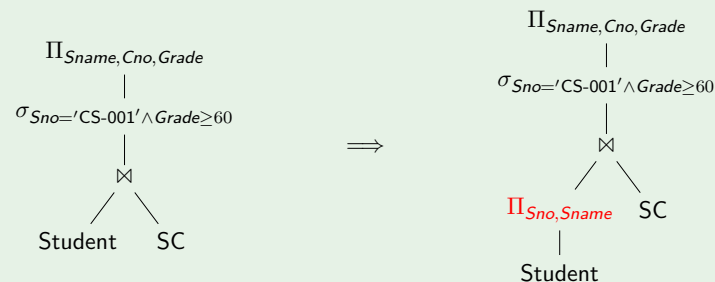


投影下推 (Projection Pushdown)

将关系代数表达式树中的投影操作下推通常可以提高查询执行效率

- 投影下推可以缩小元组的大小

例 (投影下推)

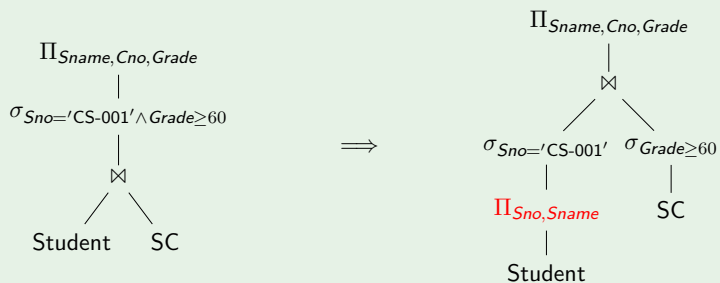


投影下推

在某些情况下, 投影下推会浪费查询优化的机会

- 原因: 投影操作的结果上没有索引

例 (投影下推)



$\Pi_{Sno, Sname}(Student)$ 上没有索引, $\sigma_{Sno=CS-001}$ 只能顺序扫描 $\Pi_{Sno, Sname}(Student)$

7.2.3 连接顺序优化

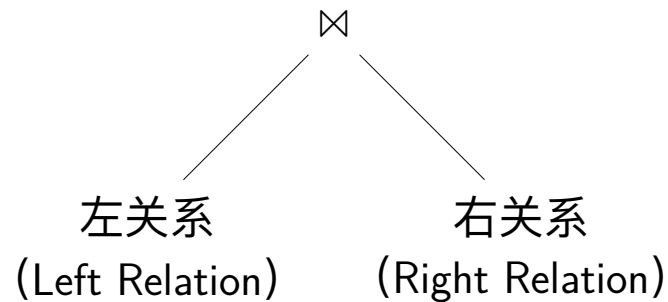
连接顺序优化 (Join Ordering)

优化连接操作的执行顺序对于提高查询执行效率至关重要

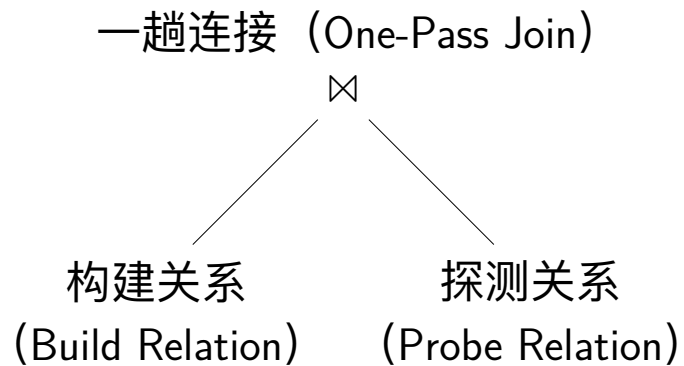
- 连接操作的执行代价高
- 在数据库上执行几十个关系的连接很常见，如联机分析处理 (Online Analytical Processing, OLAP) 任务
- 不同连接顺序的执行代价差异巨大

连接关系的角色 (Role)

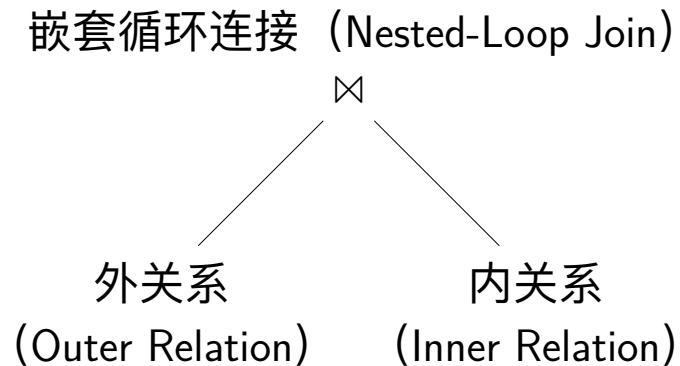
尽管连接操作满足交换律，但在物理查询计划中，连接操作的不同输入具有不同的作用



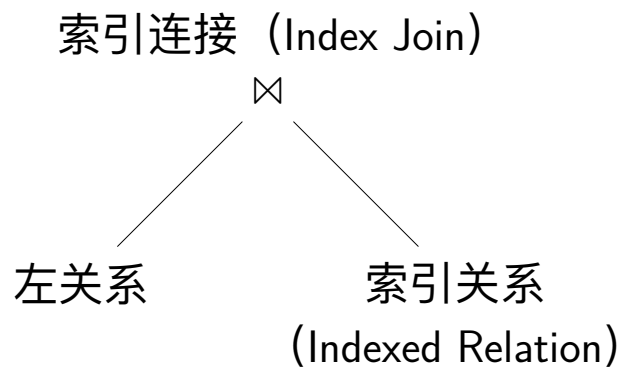
一趟连接的左关系和右关系



嵌套循环连接的左关系和右关系



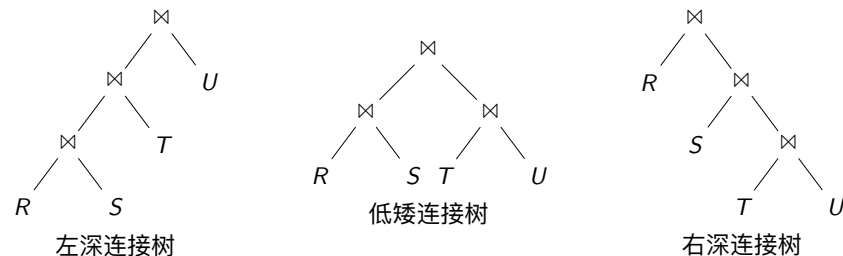
索引连接的左关系和右关系



连接树 (Join Tree)

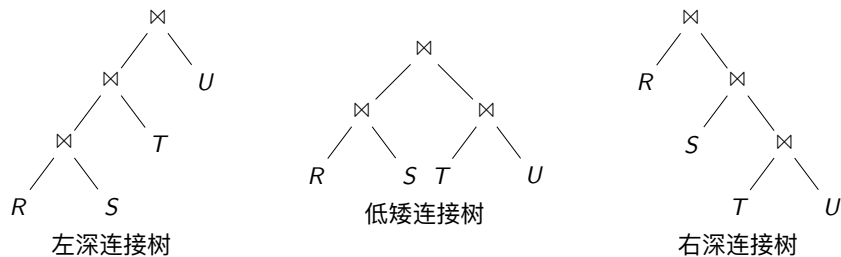
一组关系上的连接操作执行顺序可以用连接树表示

- 左深连接树 (Left-Deep Join Tree): 只有一个关系是左关系, 其他关系都是右关系
- 右深连接树 (Right-Deep Join Tree): 只有一个关系是右关系, 其他关系都是左关系
- 低矮连接树 (Bushy Tree): 除左深连接树和右深连接树以外的其他连接树



连接树 (Join Tree)

- 在逻辑查询优化阶段, 连接树的形态并不重要
- 规定连接树的形态, 是为了方便后续生成物理查询计划



左深连接树 (Left-Deep Join Tree)

System R 的查询优化器只考虑左深连接树

优点 1: 在一组给定的关系上, 左深连接树比全部连接树少得多

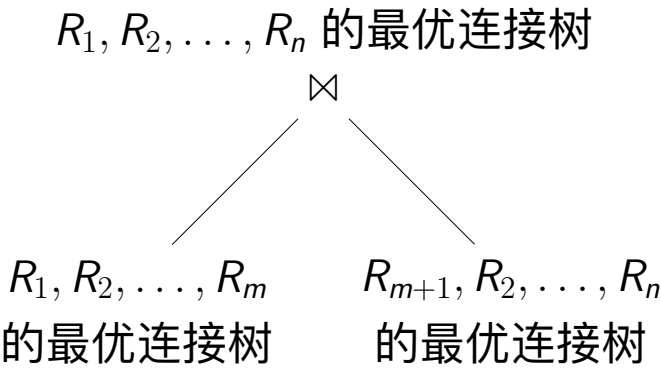
- $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$ 的全部左深连接树的数量为 $n!$
- $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$ 的全部连接树的数量为 $n!C_n$, 其中 C_n 表示含有 n 个叶子节点的树结构的数量 (Catalan 数)

$$C_1 = 1,$$
$$C_n = \sum_{i=1}^{n-1} C_i C_{n-i} \quad \text{if } n > 1$$

优点 2: 基于左深连接树的查询计划通常比基于其他类型连接树的查询计划执行效率更高

- 可能形成完全通畅的流水线 (Fully Pipelined Plan)

基于动态规划的连接顺序优化



基于动态规划的连接顺序优化

例 (基于动态规划的连接顺序优化)

为 $R(a, b) \bowtie S(b, c) \bowtie T(c, d) \bowtie U(d, a)$ 确定最优连接顺序

$R(a, b)$	$S(b, c)$	$T(c, d)$	$U(d, a)$
$T(R) = 1000$	$T(S) = 1000$	$T(T) = 1000$	$T(U) = 1000$
$V(R, a) = 100$			$V(U, a) = 50$
$V(R, b) = 200$	$V(S, b) = 100$		
	$V(S, c) = 500$	$V(T, c) = 20$	
		$V(T, d) = 50$	$V(U, d) = 1000$

阶段 1: 两个关系的最优连接顺序

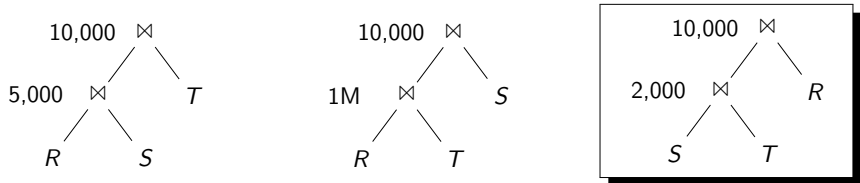
	$\{R, S\}$	$\{R, T\}$	$\{R, U\}$	$\{S, T\}$	$\{S, U\}$	$\{T, U\}$
估计基数	5,000	1M	10,000	2,000	1M	1,000
估计代价	0	0	0	0	0	0
最优计划	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

阶段 1: 两个关系的最优连接顺序

	$\{R, S\}$	$\{R, T\}$	$\{R, U\}$	$\{S, T\}$	$\{S, U\}$	$\{T, U\}$
估计基数	5,000	1M	10,000	2,000	1M	1,000
估计代价	0	0	0	0	0	0
最优计划	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

阶段 2: 三个关系的最优连接顺序

(1) $R \bowtie S \bowtie T$ 的最优连接顺序



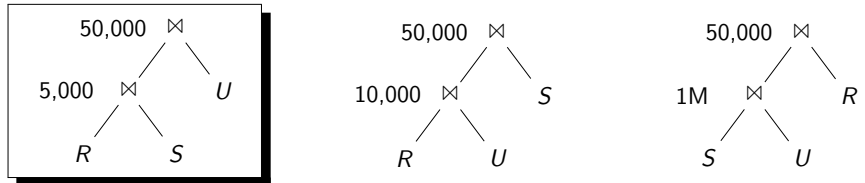
	$\{R, S, T\}$	$\{R, S, U\}$	$\{R, T, U\}$	$\{S, T, U\}$
估计基数	10,000			
估计代价	2,000			
最优计划	$(S \bowtie T) \bowtie R$			

阶段 1: 两个关系的最优连接顺序

	$\{R, S\}$	$\{R, T\}$	$\{R, U\}$	$\{S, T\}$	$\{S, U\}$	$\{T, U\}$
估计基数	5,000	1M	10,000	2,000	1M	1,000
估计代价	0	0	0	0	0	0
最优计划	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

阶段 2: 三个关系的最优连接顺序

(2) $R \bowtie S \bowtie U$ 的最优连接顺序



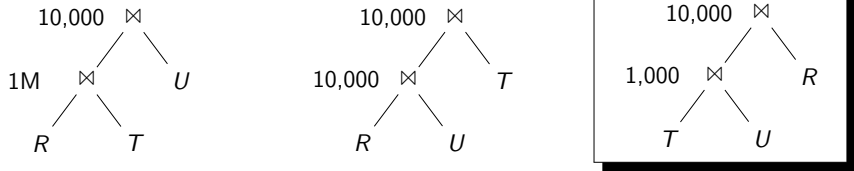
	$\{R, S, T\}$	$\{R, S, U\}$	$\{R, T, U\}$	$\{S, T, U\}$
估计基数	10,000	50,000		
估计代价	2,000	5,000		
最优计划	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$		

阶段 1：两个关系的最优连接顺序

	$\{R, S\}$	$\{R, T\}$	$\{R, U\}$	$\{S, T\}$	$\{S, U\}$	$\{T, U\}$
估计基数	5,000	1M	10,000	2,000	1M	1,000
估计代价	0	0	0	0	0	0
最优计划	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

阶段 2：三个关系的最优连接顺序

(3) $R \bowtie T \bowtie U$ 的最优连接顺序



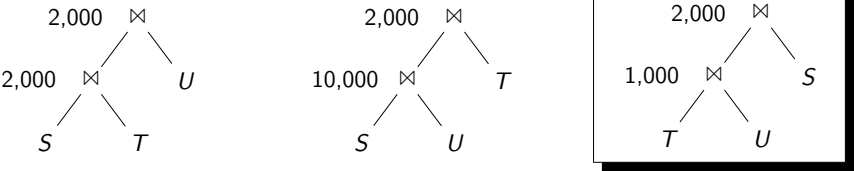
	$\{R, S, T\}$	$\{R, S, U\}$	$\{R, T, U\}$	$\{S, T, U\}$
估计基数	10,000	50,000	10,000	
估计代价	2,000	5,000	1,000	
最优计划	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	

阶段 1：两个关系的最优连接顺序

	$\{R, S\}$	$\{R, T\}$	$\{R, U\}$	$\{S, T\}$	$\{S, U\}$	$\{T, U\}$
估计基数	5,000	1M	10,000	2,000	1M	1,000
估计代价	0	0	0	0	0	0
最优计划	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

阶段 2：三个关系的最优连接顺序

(4) $S \bowtie T \bowtie U$ 的最优连接顺序



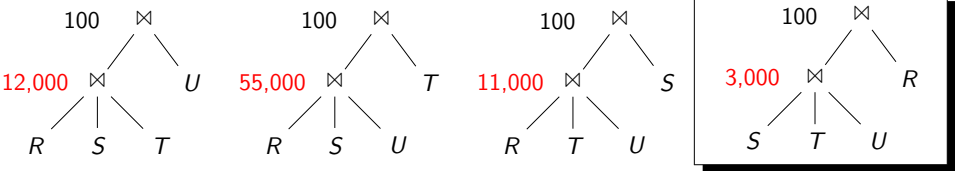
	$\{R, S, T\}$	$\{R, S, U\}$	$\{R, T, U\}$	$\{S, T, U\}$
估计基数	10,000	50,000	10,000	2,000
估计代价	2,000	5,000	1,000	1,000
最优计划	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$

阶段 2：三个关系的最优连接顺序

	$\{R, S, T\}$	$\{R, S, U\}$	$\{R, T, U\}$	$\{S, T, U\}$
估计基数	10,000	50,000	10,000	2,000
估计代价	2,000	5,000	1,000	1,000
最优计划	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$

阶段 3：四个关系的最优连接顺序

(1) 最优左深连接顺序 $(* \bowtie * \bowtie *) \bowtie *$



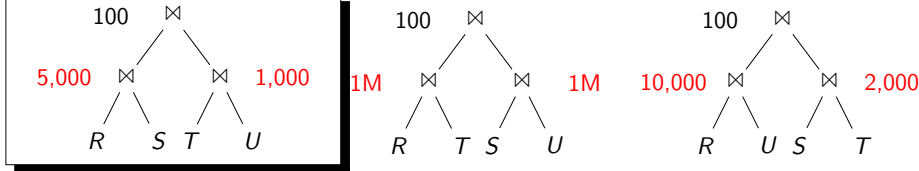
	$\{R, S, T, U\}$ (左深)	$\{R, S, T, U\}$ (Bushy)	$\{R, S, T, U\}$ (全部)
估计基数	100		
估计代价	3,000		
最优计划	$((T \bowtie U) \bowtie S) \bowtie R$		

阶段 1：两个关系的最优连接顺序

	$\{R, S\}$	$\{R, T\}$	$\{R, U\}$	$\{S, T\}$	$\{S, U\}$	$\{T, U\}$
估计基数	5,000	1M	10,000	2,000	1M	1,000
估计代价	0	0	0	0	0	0
最优计划	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

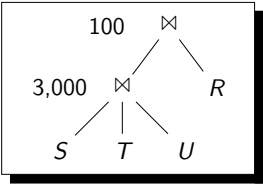
阶段 3：四个关系的最优连接顺序

(2) 最优 Bushy 连接顺序 $(* \bowtie *) \bowtie (* \bowtie *)$



	$\{R, S, T, U\}$ (左深)	$\{R, S, T, U\}$ (Bushy)	$\{R, S, T, U\}$ (全部)
估计基数	100	100	
估计代价	3,000	6,000	
最优计划	$((T \bowtie U) \bowtie S) \bowtie R$	$(R \bowtie S) \bowtie (T \bowtie U)$	

阶段 3：四个关系的最优连接顺序
(3) 全局最优连接顺序



	$\{R, S, T, U\}$ (左深)	$\{R, S, T, U\}$ (Bushy)	$\{R, S, T, U\}$ (全部)
估计基数	100	100	100
估计代价	3,000	6,000	3,000
最优计划	$((T \bowtie U) \bowtie S) \bowtie R$	$(R \bowtie S) \bowtie (T \bowtie U)$	$((T \bowtie U) \bowtie S) \bowtie R$

RDBMS 对连接顺序优化的控制

- PostgreSQL：禁用连接顺序优化（外连接除外）
`SET join_collapse_limit = 1`
- DuckDB：禁用连接顺序优化
`SET join_order = 'force_physical';`
- DuckDB：启用最优连接顺序
`SET join_order = 'optimal';`
- DuckDB：强制使用左深连接树
`SET join_order = 'left_deep';`

7.3 物理查询优化

物理查询优化（Physical Query Optimization）

- ① 物理查询计划枚举
- ② 物理查询计划代价估计

物理查询计划枚举

- ❶ 为逻辑查询计划中每个逻辑算子选择合适的物理算子
- ❷ 为物理查询计划指定合适的执行模型，即每个算子如何把结果传递给下一个算子

物理选择算子的确定

确定物理算子本质上是选择最高效的存取路径 (Access Path)

- ❶ 索引扫描 (Index Scan) + 过滤 (Filtering)
- ❷ 多索引扫描 + 求交集
- ❸ 仅用索引 (Index-Only)
- ❹ 顺序扫描 (Sequential Scan)

7.3.1 物理算子的确定

索引扫描 (Index Scan) + 过滤 (Filtering)

$\sigma_{\theta_1 \wedge \theta_2}(R)$ ，其中 θ_1 和 θ_2 为两个选择条件

适用条件

- 关系 R 上建有索引 I ，支持条件 θ_1 上的查找

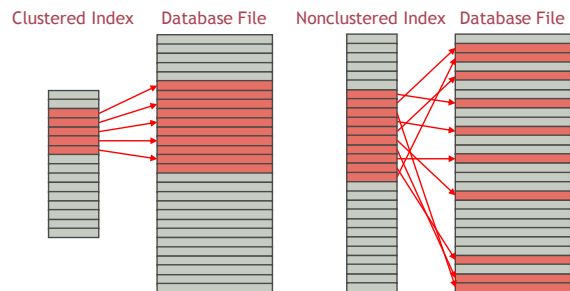
存取路径

- 在索引 I 上查找满足条件 θ_1 的元组的 RID
- 根据元组 RID，从 R 的文件中取出元组
- 使用条件 θ_2 对元组进行过滤

索引扫描 + 过滤

“索引扫描 + 过滤”的存取路径在下面两个情况下非常有效

- 索引 I 是聚簇索引 (Clustered Index)
- 索引 I 是非聚簇索引 (Nonclustered Index)，但只有少量元组满足条件 θ_1



多索引扫描 + 求交集

$\sigma_{\theta_1 \wedge \theta_2}(R)$ ，其中 θ_1 和 θ_2 为两个选择条件

适用条件

- 关系 R 上建有索引 I_1 ，支持条件 θ_1 上的查找
- 关系 R 上建有索引 I_2 ，支持条件 θ_2 上的查找

存取路径

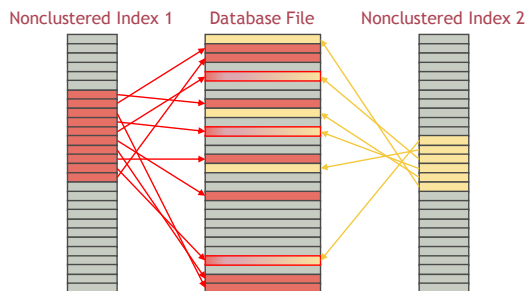
- 在索引 I_1 上查找满足条件 θ_1 的元组的 RID
- 在索引 I_2 上查找满足条件 θ_2 的元组的 RID
- 计算两个元组 RID 集合的交集
- 根据交集的元组 RID，从 R 的文件中取出元组

该存取路径在 MySQL 中称作**索引合并 (Index Merge)**

多索引扫描 + 求交集

“多索引扫描 + 求交集”的存取路径在下面情况下非常有效

- 索引 I_1 和 I_2 都是非聚簇索引
- 满足条件 θ_1 的元组和满足条件 θ_2 的元组均较多
- 但同时满足条件 θ_1 和 θ_2 的元组较少



仅用索引 (Index-Only)

$\sigma_{\theta}(R)$

- θ ：选择条件
- L ：查询计划中该选择操作的后续操作所涉及的 R 的属性集合

适用条件

- 关系 R 上建有索引 I ，支持条件 θ 上的查找
- 索引 I 是**覆盖索引 (Covering Index)**，其中包含 L 中所有属性

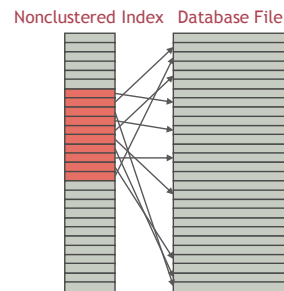
存取路径

- 在索引 I 上查找满足条件 θ 的索引键
- 将索引键向 L 上做投影

仅用索引

“仅用索引”的存取路径在下面的情况非常有效

- 关系 R 上的聚簇索引不支持选择条件 θ
- 索引 I 是覆盖索引



顺序扫描 (Sequential Scan)

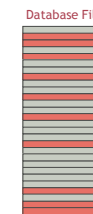
$\sigma_{\theta}(R)$, 其中 θ 为选择条件

适用条件

- 关系 R 上没有索引可以支持条件 θ 上的查找

存取路径

- 顺序扫描 R 中每条元组
- 使用条件 θ 对元组进行过滤



物理连接算子的确定

- 1 一趟连接 (One-Pass Join)
- 2 索引连接 (Index Join)
- 3 排序归并连接 (Sort-Merge Join)
- 4 哈希连接 (Hash Join)
- 5 嵌套循环连接 (Nested-Loop Join)

一趟连接 (One-Pass Join)

适用条件：左关系可以全部读入缓冲池

索引连接 (Index Join)

适用条件

- 左关系较小
- 右关系在连接属性上建有索引

排序归并连接 (Sort-Merge Join)

适用条件：

- 至少有一个关系已经按连接属性排序
- 多个关系在相同连接属性上做多路连接也适合使用排序归并连接，如 $R(a, b) \bowtie S(a, c) \bowtie T(a, d)$

哈希连接 (Hash Join)

适用条件：当一趟连接、排序归并连接、索引连接都不适用时，哈希连接总是好的选择

嵌套循环连接 (Nested-Loop Join)

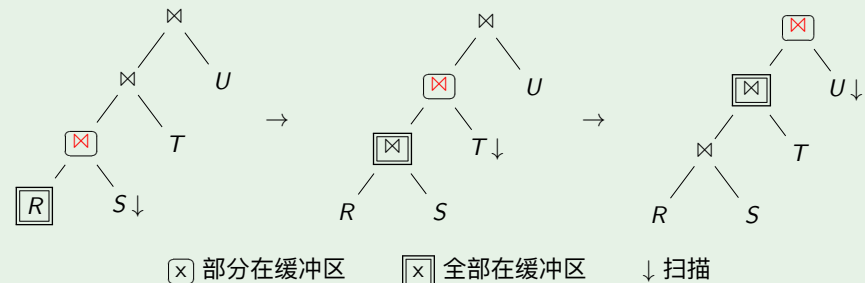
适用条件：当可用的缓冲区页面非常少时，可使用嵌套循环连接

7.3.2 数据传递方式的确定

System R 选择左深连接树的原因

如果下列查询计划全部使用一趟连接（One-pass Join）算法，则在流水线查询执行模型下，左深连接树查询计划在任何时刻对内存的需求都比其他查询计划更低

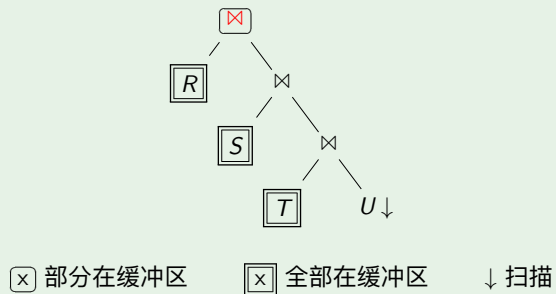
例 (左深连接树)



System R 选择左深连接树的原因

如果下列查询计划全部使用一趟连接（One-pass Join）算法，则在流水线查询执行模型下，左深连接树查询计划在任何时刻对内存的需求都比其他查询计划更低

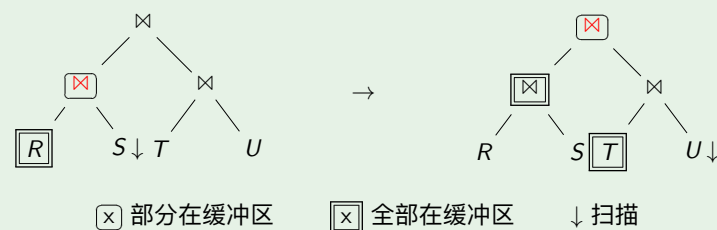
例 (右深连接树)



System R 选择左深连接树的原因

如果下列查询计划全部使用一趟连接（One-pass Join）算法，则在流水线查询执行模型下，左深连接树查询计划在任何时刻对内存的需求都比其他查询计划更低

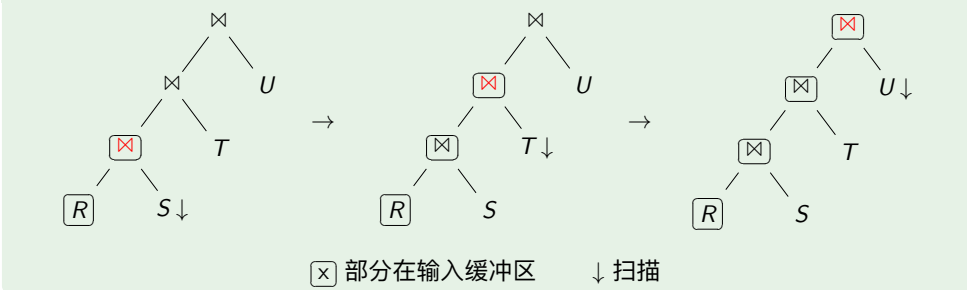
例 (Bushy 连接树)



System R 选择左深连接树的原因

如果下列查询计划全部使用嵌套循环连接（Nested-Loop Join）算法，则在流水线查询执行模型下，左深连接树查询计划不需要多次构建中间关系

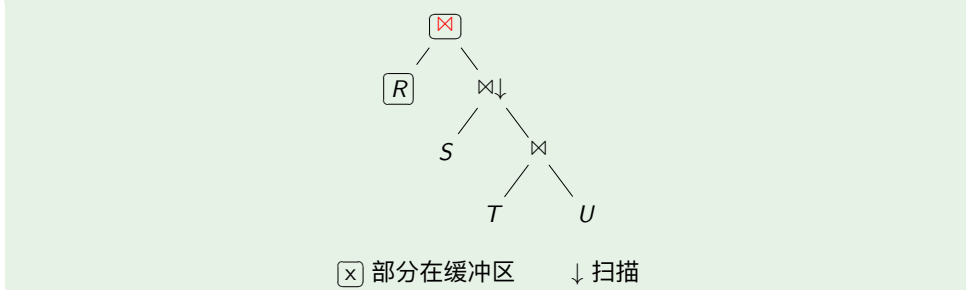
例 (左深连接树)



System R 选择左深连接树的原因

如果下列查询计划全部使用嵌套循环连接（Nested-Loop Join）算法，则在流水线查询执行模型下，左深连接树查询计划不需要多次构建中间关系

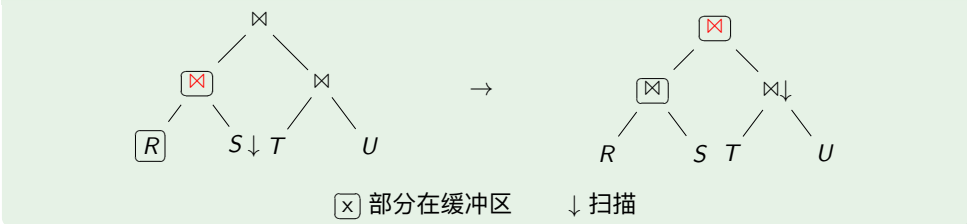
例 (右深连接树)



System R 选择左深连接树的原因

如果下列查询计划全部使用嵌套循环连接（Nested-Loop Join）算法，则在流水线查询执行模型下，左深连接树查询计划不需要多次构建中间关系

例 (Bushy 连接树)



总结

- 1 7.1 查询优化的基本概念
- 2 7.2 逻辑查询优化
 - 7.2.1 逻辑查询计划的代价估计
 - 7.2.2 逻辑查询计划枚举
 - 7.2.3 连接顺序优化
- 3 7.3 物理查询优化
 - 7.3.1 物理算子的确定
 - 7.3.2 数据传递方式的确定